Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O enunciado solicita a implementação das funções de sincronização de uma estação de metrô em C, na qual o vagão e os passageiros são threads independentes. A entrega deve usar pthreads, no máximo um lock por estação, sem semáforos e sem busy-wait, além de um Makefile que gere o executável sync_car.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Implementação em C e executável em Linux, Unix ou macOS	OK	Compilado com GCC 16.2.1 e Clang 23.1.3; Linux testado. Código usa apenas POSIX.
station_init inicializa mutex, condições e contadores	OK	sync_car.c:22-34; build e execução do runner.
station_load_car retorna imediatamente quando não há passageiros	OK	sync_car.c:50-54; runner linhas 77-80; log linhas 439-452.
station_load_car retorna imediatamente quando count == 0	OK	sync_car.c:50-54; harness de caso limite; log linha 457.
station_load_car só retorna depois que todos os passageiros chamam on_board	OK	sync_car.c:59-61; condição boarded < seats_taken; 200 execuções (log linha 909).
station_wait_for_car bloqueia até haver vagão e assento livre	OK	sync_car.c:75-80; 100 threads de passageiros do runner.
station_on_board informa ao vagão que o passageiro embarcou	OK	sync_car.c:91-92; incrementa boarded e sinaliza.
Usar as 7 funções pthread exigidas	OK	pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock, pthread_cond_init, pthread_cond_signal, pthread_cond_wait e pthread_cond_broadcast; grep no log linhas 475-481.
Não usar semáforos nem outras primitivas de sincronização	OK	Somente mutex e variáveis de condição; grep no log linhas 482-484.
No máximo um lock por struct station	OK	Existe exatamente um pthread_mutex_t em sync_car.h.
Assumir um carro por vez e todos no mesmo destino	OK	Premissa do enunciado; a implementação não mantém fila de destinos.
Permitir que vários passageiros embarquem simultaneamente	OK	pthread_cond_broadcast em sync_car.c:57; 100 threads do runner.
Não usar busy-wait	OK	Todas as esperas usam pthread_cond_wait; não há laço de polling.
README + Makefile	OK	make clean e make;
Relatório	OK	

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

Saída esperada: a última linha deve ser Looks good! e o código de retorno deve ser 0. O runner imprime as chegadas e partidas dos vagões.

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. sync_car.h → define struct station (um mutex, duas condições e quatro contadores) e os protótipos das funções. A struct fica no header porque o runner declara struct station station; na pilha e precisa da definição completa.
2. sync_car.c → implementa station_init, station_load_car, station_wait_for_car e station_on_board, além dos helpers de validação.
3. sync_car-runner.c → testador fornecido pelo professor; contém o main. Não foi modificado.
4. Makefile → compila os dois arquivos .c com -std=gnu11 -pthread, gera sync_car e tem o alvo clean.

Decisões técnicas:

1. Um único mutex protege todos os campos da estação → evita corridas entre contadores e evita deadlock por ordem de locks → todo acesso ao estado ocorre dentro da região crítica.
2. Duas variáveis de condição: car_arrived para passageiros e passenger_boarded para o vagão → separa os dois motivos de espera → cada grupo acorda apenas quando faz sentido.
3. seats_taken e boarded são contadores distintos → seats_taken indica assento reservado e boarded indica embarque confirmado → o vagão não parte antes de todos os on_board.
4. pthread_cond_broadcast na chegada do vagão → acorda vários passageiros que disputam os assentos → atende ao requisito de embarque simultâneo.
5. pthread_cond_signal no on_board → só a thread do vagão espera nessa condição → um sinal é suficiente.
6. capacity = 0 no fim do load_car → fecha as portas do vagão → passageiros que chegam depois esperam o próximo vagão.
7. Retorno imediato para count == 0 e waiting == 0 → sem assentos ou sem passageiros não há o que esperar → evita deadlock e atende ao runner.
8. Validação de NULL e dos retornos pthread → check_station e check_pthread falham rápido com EXIT_FAILURE → erros de contrato não ficam silenciosos.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Mutex único + duas condições + contadores	Implementação da estação: passageiros esperam assento; vagão espera confirmações de embarque.	Não houve troca de estratégia. A mesma abordagem foi refinada com tratamento de erros, README e validação final.
S2			
S3			

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Rodar make clean e make com o Makefile inicial.	Falha	As receitas estavam com espaços em vez de tabs; o Makefile apontou missing separator.	2026-11-03 22:29	evidencias.log linhas 9-13 e 16-29; Figura 1.
2	S1	Criar a estrutura (header, Makefile e runner) e compilar as funções ainda vazias.	Parcial	Warnings de parâmetros não usados porque as funções eram stubs.	2026-11-03 22:44	evidencias.log linhas 41-66; commit 6c31464.
3	S1	Implementar station_init com mutex, condições e contadores zerados.	OK		2026-11-03 22:51	Commit 78c548a.
4	S1	Implementar station_wait_for_car e station_on_board.	OK		2026-11-03 23:23	Commit 3cf4fa7.
5	S1	Implementar station_load_car e rodar o testador.	OK		2026-11-03 23:50	evidencias.log linhas 107-116 (primeiro Looks good!); commit 757587e.
6	S1	Adicionar tratamento de erro e validação de NULL/pthread.	Parcial	O caminho count < 0 chamava pthread_mutex_unlock sem ter feito o lock; detectado em revisão.	2026-11-03 23:58	Commit 332ad8c; correção no commit 0ece822.
7	S1	Completar o README com descrição, ambiente, arquivos, compilação e testes.	OK		2026-11-04 17:29	Commit 36c3ed8.
8	S1	Corrigir o tratamento de erro e fazer a validação final (build, runner, caso limite/inválido).	OK		2026-11-04 17:30 (correção) / 17:59 (validação)	Commit 0ece822; evidencias.log linhas 422-487; Figuras 2 e 3.

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

6.1 Log automático

O arquivo evidencias.log foi gerado com script -a e contém três sessões: a sessão de desenvolvimento (2026-11-03 22:29:35 a 2026-11-04 00:01:01), a sessão de validação final (2026-11-04 17:59:44 a 17:59:45) e a sessão de estresse (2026-11-04 18:07:28 a 18:09:22).

Trechos relevantes:

- linhas 9-29: erro de missing separator no Makefile e o cat -A que mostra as receitas com espaços;
- linhas 107-116: primeira execução completa do runner terminando em Looks good!;
- linhas 422-487: sessão final, com date, whoami, pwd, build, execução do runner, harness de caso limite/inválido e verificação das chamadas pthread;
- linhas 488-1336: sessão de estresse, com 200 execuções GCC (linha 909), 100 execuções Clang (linha 1119) e 100 execuções com -O2 (linha 1329).

6.2 Prints

Os prints abaixo são recortes renderizados do log e da saída real, sem os códigos de controle do terminal. Não são mockups.



Figura 1 — Recorte do evidencias.log (linhas 9-29): erro principal do Makefile. O cat -A mostra as receitas indentadas com espaços e o $ no fim das linhas; faltava o tab que o Make exige.

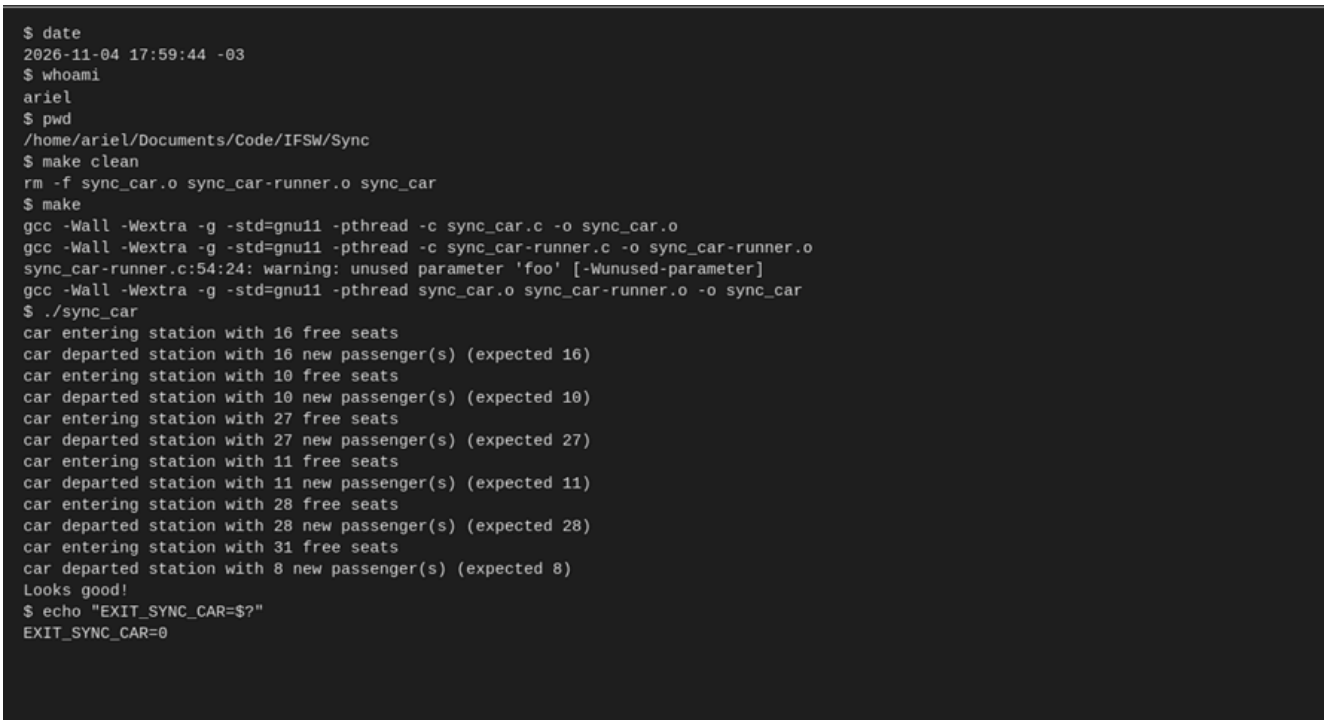


Figura 2 — Recorte da sessão final (linhas 422-454): make clean, make e ./sync_car terminando em Looks good! com EXIT_SYNC_CAR=0. O warning de foo é do runner fornecido, não do sync_car.c.



Figura 3 — Recorte da sessão final (linhas 455-462): harness de caso limite (count=0) e inválido (count=-1). O passageiro permanece bloqueado nos dois casos e embarca normalmente quando chega um vagão com count=1.

7. Uso de IA

Onde usei IA (2–3 linhas):
Usei IA como apoio para entender a implementação, discutir riscos de concorrência e portabilidade. A versão final do código e as execuções foram validadas manualmente.

Prompts principais (sem história):

- Leia o enunciado e explique os requisitos, restrições e critérios de entrega.
- Revise o `sync_car.c` e aponte possíveis erros de gcc ou diferença de versão.
- Monte um checklist de entrega e verifique se o Makefile e o README atendem ao enunciado.

O que validei manualmente e como:
Compilei com `make clean && make`, executei `./sync_car` e conferi `Looks good!` e exit 0, rodei 200 vezes com GCC, 100 vezes com Clang e 100 vezes com `-O2`, conferi as chamadas pthread e registrei tudo no `evidencias.log` (linhas 422-1336).

8. Reflexão Final (5–8 linhas)

Reflexão:

Teria organizado melhor meu tempo para deixar o processo de produção mais leve apesar de não ter sido tão complexa para saber exatamente o que deveria ser feito. Assim, conseguiria desenvolver um trabalho mais organizado e evitar alguns erros simples, como problemas bobos. Além disso, tentaria testar estratégias diferentes durante o desenvolvimento, analisando o que funcionou melhor e o que poderia ser melhorado. Dessa forma, acredito que conseguiria trabalhar com mais calma e chegar a um resultado melhor que o atual.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Com mais tempo disponível, procuraria testar uma variedade maior de possíveis erros e situações inesperadas, buscando identificar falhas que talvez não tenham sido percebidas durante o desenvolvimento inicial. Além disso, tentaria ampliar e aprimorar o tratamento de erros, caso fosse necessário, para tornar a implementação mais completa e preparada para diferentes cenários. Também buscaria explorar outras possibilidades de implementação da atividade e, se possível, desenvolver versões utilizando as funções que não eram permitidas, com o objetivo de comparar as diferenças de desempenho, complexidade e facilidade de implementação entre as abordagens. Dessa forma, seria possível compreender melhor as vantagens e desvantagens de cada método e identificar quais soluções seriam mais eficientes em diferentes situações.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.